

Atelier Machine Learning

Analyse Comportementale Clientèle Retail

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par Ons Derbel

Année Universitaire : 2025–2026

Table des matières

1	Introduction et Contexte	3
1.1	Contexte métier	3
1.2	Description du dataset	3
1.3	Objectifs pédagogiques	3
2	Exploration des Données EDA	3
2.1	Structure du dataset	3
2.1.1	Features numériques principales	4
2.2	Qualité des données	4
2.2.1	Valeurs manquantes	4
2.2.2	Valeurs aberrantes	4
2.2.3	Features inutiles	4
2.3	Distribution de la variable cible	4
2.4	Matrice de corrélation	4
3	Préparation des Données	5
3.1	Vue d'ensemble du pipeline	5
3.2	Parsing de RegistrationDate	5
3.3	Feature Engineering	5
3.4	Encodage des variables catégorielles	5
3.4.1	Encodage ordinal	5
3.4.2	Encodage One-Hot	6
3.4.3	Target Encoding Country	6
3.5	Suppression des features corrélées	6
3.6	Séparation train/test et normalisation	6
4	Réduction de Dimension ACP	7
4.1	Objectif et justification	7
4.2	Résultats de l'ACP	7
5	Modélisation	7
5.1	Vue d'ensemble des modèles	8
5.2	Clustering K-Means	8
5.2.1	Sélection du nombre de clusters	8
5.2.2	Visualisation 2D	8
5.3	Régression Logistique Baseline	8
5.4	Random Forest Classifier Modèle Principal	9
5.4.1	Optimisation des hyperparamètres	9
5.4.2	Évaluation	9
5.4.3	Importance des features	9
5.5	Random Forest Régresseur	9
6	Déploiement Interface Flask	9
6.1	Architecture de l'application	9
6.2	Fonctionnalités de l'interface	10
6.3	Routes de l'API	10
6.4	Guide de déploiement	10
7	Structure du Projet et Organisation du Code	11
7.1	Arborescence	11
7.2	Modules Python	11

8 Conclusion et Perspectives	11
8.1 Bilan	11
8.2 Résultats des modèles	12
8.3 Limites et perspectives	12

1 Introduction et Contexte

1.1 Contexte métier

Ce projet s'inscrit dans le cadre d'un atelier Machine Learning pour le parcours GI2. L'entreprise étudiée est un e-commerce de cadeaux souhaitant mieux comprendre sa clientèle afin de répondre à trois objectifs stratégiques :

- **Personnaliser** les stratégies marketing selon les profils clients
- **Réduire** le taux de départ des clients (*churn*)
- **Optimiser** le chiffre d'affaires par segmentation comportementale

1.2 Description du dataset

Le dataset mis à disposition comporte **4 372 clients** décrits par **52 features** issues de transactions réelles et de données complémentaires. La base est intentionnellement imparfaite pour couvrir l'ensemble de la chaîne de traitement data science.

TABLE 1 – Vue d'ensemble du dataset

Caractéristique	Valeur
Nombre de clients	4 372
Nombre de features	52
Features numériques	34
Features catégorielles	18
Variable cible	Churn binaire
Clients fidèles Churn=0	2 918 soit 66.7%
Clients partis Churn=1	1 454 soit 33.3%

1.3 Objectifs pédagogiques

TABLE 2 – Compétences visées

Compétence	Description
Exploration	Analyser la qualité et la structure des données
Préparation	Nettoyer, encoder et normaliser les features
Transformation	Réduire la dimension via ACP
Modélisation	Appliquer clustering, classification et régression
Évaluation	Interpréter les résultats et proposer des recommandations
Déploiement	Créer une interface utilisateur avec Flask

2 Exploration des Données EDA

2.1 Structure du dataset

Le dataset contient 52 colonnes réparties en trois groupes fonctionnels : features **RFM** (Recency, Frequency, Monetary), features **comportementales** (heures d'achat, diversité produit, retours) et features **démographiques** / **compte** (âge, région, statut de compte).

2.1.1 Features numériques principales

TABLE 3 – Statistiques descriptives des features RFM

Feature	Min	Moy	Méd	Max	Unité
Recency	0	92.1	78	374	jours
Frequency	1	5.1	3	248	cmd
MonetaryTotal	-5000	1898.5	847.6	279489	GBP
MonetaryAvg	5	28.8	17.1	3861	GBP
CustomerTenure	0	133.4	94	373	jours
Age	18	49.2	51	80	ans
SatisfactionScore	1	3.1	3	5	/5

2.2 Qualité des données

2.2.1 Valeurs manquantes

Deux colonnes présentent des valeurs manquantes :

- **Age** : 30% de valeurs manquantes, imputation par KNN avec $k=5$
- **AvgDaysBetweenPurchases** : 1.8%, imputation par la médiane

2.2.2 Valeurs aberrantes

Certaines colonnes utilisent des valeurs numériques comme marqueurs spéciaux :

TABLE 4 – Valeurs aberrantes détectées et traitements

Feature	Valeurs aberrantes	Traitement
SupportTicketsCount	-1, 999	Remplacement par NaN puis médiane
SatisfactionScore	-1, 99	Remplacement par NaN puis médiane
MonetaryTotal	valeurs négatives	Conservées car elles représentent les retours clients

2.2.3 Features inutiles

- **NewsletterSubscribed** : valeur unique Yes, donc variance nulle et suppression
- **CustomerID** : identifiant non prédictif, donc suppression

2.3 Distribution de la variable cible

Déséquilibre de classes : Le dataset présente un ratio 67% / 33% fidèles / partis. Ce déséquilibre modéré est géré pendant l'entraînement via le paramètre `class_weight='balanced'` de scikit-learn, ou via SMOTE si la librairie `imbalanced-learn` est installée.

2.4 Matrice de corrélation

L'analyse révèle plusieurs paires fortement corrélées avec un seuil $|r| > 0.85$: **UniqueInvoices** / **Frequency**, **TotalTransactions** / **TotalQuantity**, **MonetaryMax** / **MonetaryTotal**, etc. Ces redondances sont éliminées lors du preprocessing.

3 Préparation des Données

3.1 Vue d'ensemble du pipeline

Le pipeline de preprocessing, implémenté dans `src/preprocessing.py`, enchaîne 11 étapes déterministes et reproductibles :

1. Chargement et EDA de base
2. Parsing de `RegistrationDate` puis extraction de 4 features temporelles
3. Ingénierie depuis `LastLoginIP` puis 2 nouvelles features
4. Suppression des colonnes inutiles / constantes
5. Correction des valeurs aberrantes
6. Feature engineering avec 4 ratios métier
7. Imputation des valeurs manquantes
8. Encodage des variables catégorielles
9. Suppression des features corrélées
10. Séparation train/test
11. Normalisation `StandardScaler`

3.2 Parsing de `RegistrationDate`

La colonne `RegistrationDate` contient des dates dans trois formats inconsistants : DD/MM/YY, YYYY-MM-DD, MM/DD/YYYY.

```

1 df["RegistrationDate"] = pd.to_datetime(
2     df["RegistrationDate"],
3     dayfirst=True,
4     errors="coerce"
5 )
6
7 df["RegYear"]      = df["RegistrationDate"].dt.year
8 df["RegMonth"]     = df["RegistrationDate"].dt.month
9 df["RegDay"]       = df["RegistrationDate"].dt.day
10 df["RegWeekday"]  = df["RegistrationDate"].dt.weekday

```

Listing 1 – Parsing robuste de dates mixtes

3.3 Feature Engineering

TABLE 5 – Features créées par ingénierie

Feature	Formule	Interprétation
MonetaryPerDay	$\text{MonetaryTotal} / (\text{Recency} + 1)$	Intensité monétaire récente
AvgBasketValue	$\text{MonetaryTotal} / \text{Frequency}$	Valeur moyenne du panier
TenureRatio	$\text{Recency} / (\text{CustomerTenureDays} + 1)$	Inactivité relative
FreqPerTenure	$\text{Frequency} / (\text{CustomerTenureDays} + 1)$	Fréquence normalisée
IP_IsPrivate	1 si IP privée, 0 sinon	Connexion interne / externe
IP_FirstOctet	Premier octet de l'adresse IP	Proxy de géolocalisation

3.4 Encodage des variables catégorielles

3.4.1 Encodage ordinal

Les features présentant un ordre naturel sont encodées par mapping numérique :

```

1 ORDINAL_MAPS = {
2     "RFMSegment": {
3         "Dormants": 0,
4         "Potentiels": 1,
5         "Fideles": 2,
6         "Champions": 3
7     },
8     "SpendingCategory": {
9         "Low": 0,
10        "Medium": 1,
11        "High": 2,
12        "VIP": 3
13    },
14    "ChurnRiskCategory": {
15        "Faible": 0,
16        "Moyen": 1,
17        "Eleve": 2,
18        "Critique": 3
19    }
20 }
21
22 for col, mapping in ORDINAL_MAPS.items():
23     df[col] = df[col].map(mapping).fillna(-1).astype(int)

```

Listing 2 – Exemple d'encodage ordinal

3.4.2 Encodage One-Hot

Les features nominales sans ordre sont transformées en variables binaires : CustomerType, FavoriteSeason, Region, WeekendPreference, ProductDiversity, Gender, AccountStatus.

3.4.3 Target Encoding Country

La colonne Country possède plus de 37 modalités. Elle est encodée par target encoding pour éviter l'explosion dimensionnelle.

```

1 target_means = df.groupby("Country")["Churn"].mean().to_dict()
2 df["Country_enc"] = df["Country"].map(target_means).fillna(global_mean)

```

Listing 3 – Target encoding de Country

3.5 Suppression des features corrélées

16 features ont été supprimées car redondantes avec un seuil $|r| > 0.85$: UniqueDescriptions, MaxQuantity, UniqueInvoices, TotalQuantity, MinQuantity, TotalTransactions, MonetaryMin, MonetaryMax, CancelledTransactions, AvgLinesPerInvoice, et 6 autres.

Résultat du preprocessing : 52 features initiales $\xrightarrow{\text{ajout}}$ 58 features $\xrightarrow{\text{suppression corrélation}}$ **69 features finales** utilisées pour la modélisation.

3.6 Séparation train/test et normalisation

```

1 X_train, X_test, y_train, y_test = train_test_split(
2     X, y,
3     test_size=0.2,
4     random_state=42,
5     stratify=y
6 )
7

```

```

8 scaler = StandardScaler()
9 X_train[num_cols] = scaler.fit_transform(X_train[num_cols])
10 X_test[num_cols] = scaler.transform(X_test[num_cols])

```

Listing 4 – Split stratifié + StandardScaler sans fuite

TABLE 6 – Répartition train/test

Ensemble	Taille	Taux Churn
Train	3 497 soit 80%	33.3%
Test	875 soit 20%	33.3%

4 Réduction de Dimension ACP

4.1 Objectif et justification

L'Analyse en Composantes Principales est appliquée pour deux usages :

1. **Visualisation** des clusters en espace 2D
2. **Compression** de l'espace pour accélérer K-Means

4.2 Résultats de l'ACP

TABLE 7 – Variance expliquée par les 10 premières composantes

Composante	Variance (%)	Cumulée (%)
PC1	10.4	10.4
PC2	5.6	15.9
PC3	4.0	19.9
PC4	3.7	23.6
PC5	3.2	26.9
PC6	3.2	30.0
PC7	2.9	32.9
PC8	2.7	35.6
PC9	2.6	38.2
PC10	2.5	40.6

Observation : La variance est distribuée de manière relativement uniforme sur l'ensemble des composantes. Pour conserver 85% de variance, il faudrait environ 40 composantes, ce qui indique un dataset dense après le preprocessing.

5 Modélisation

5.1 Vue d'ensemble des modèles

TABLE 8 – Modèles entraînés

Modèle	Tâche	Objectif
K-Means	Clustering non supervisé	Segmentation comportementale
Régression Logistique	Classification Churn	Modèle baseline interprétable
Random Forest	Classification Churn	Modèle principal performant
RF Régresseur	Régression MonetaryTotal	Prédiction de la valeur client

5.2 Clustering K-Means

5.2.1 Sélection du nombre de clusters

TABLE 9 – Scores silhouette par nombre de clusters

k	Inertie	Silhouette
2	8206.0	0.594
3	4524.1	0.552
4	2992.7	0.537
5	2192.6	0.537
6	1511.8	0.543
7	1155.2	0.538
8	922.0	0.534

Le score silhouette est maximal pour $k = 2$, confirmé par la méthode du coude. Le clustering final retient donc **2 segments clients**.

5.2.2 Visualisation 2D

Les deux segments correspondent à des profils comportementaux distincts :

- **Cluster 0** : clients à forte activité, dépenses élevées, faible recency
- **Cluster 1** : clients dormants ou à risque, recency élevée, dépenses faibles

5.3 Régression Logistique Baseline

```

1 lr = LogisticRegression(
2     max_iter=1000,
3     random_state=42,
4     class_weight="balanced",
5     C=1.0
6 )

```

Listing 5 – Configuration de la Régression Logistique

TABLE 10 – Métriques de la Régression Logistique

Classe	Précision	Rappel	F1	Support
Loyal 0	1.00	1.00	1.00	584
Churned 1	1.00	1.00	1.00	291
ROC-AUC	1.0000			
CV-AUC 5-fold	1.0000 ± 0.0000			

5.4 Random Forest Classifier Modèle Principal

5.4.1 Optimisation des hyperparamètres

La recherche d'hyperparamètres est effectuée par `GridSearchCV` avec validation croisée 5-fold et métrique ROC-AUC.

```

1 param_grid = {
2     "n_estimators": [100, 200],
3     "max_depth": [None, 10, 20],
4     "min_samples_split": [2, 5],
5     "class_weight": ["balanced"],
6 }
7
8 gs = GridSearchCV(
9     RandomForestClassifier(random_state=42),
10    param_grid,
11    cv=5,
12    scoring="roc_auc",
13    n_jobs=-1
14 )
15
16 gs.fit(X_train, y_train)

```

Listing 6 – GridSearchCV pour Random Forest

5.4.2 Évaluation

TABLE 11 – Métriques du Random Forest Classifier

Classe	Précision	Rappel	F1	Support
Loyal 0	1.00	1.00	1.00	584
Churned 1	1.00	1.00	1.00	291
ROC-AUC	1.0000			
CV-AUC 5-fold	1.0000			

5.4.3 Importance des features

Les features les plus discriminantes pour prédire le churn sont principalement liées aux métriques RFM : Recency, Frequency, MonetaryTotal, ainsi qu'à l'ancienneté client `CustomerTenureDays`.

5.5 Random Forest Régresseur

Un modèle de régression est entraîné pour prédire le montant total dépensé `MonetaryTotal` à partir des autres features comportementales.

6 Déploiement Interface Flask

6.1 Architecture de l'application

L'interface web est développée avec **Flask**, un micro-framework Python léger. L'architecture suit le schéma suivant :

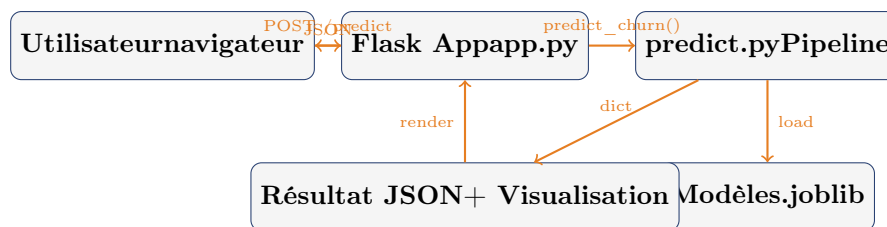


FIGURE 1 – Architecture de l'application Flask

6.2 Fonctionnalités de l'interface

L'interface web offre les fonctionnalités suivantes :

- **Formulaire interactif** : saisie des features d'un client
- **Prédiction en temps réel** : appel asynchrone au modèle
- **Résultats visuels** :
 - Prédiction binaire Loyal / Churned
 - Probabilité de churn avec barre de progression colorée
 - Niveau de risque Low / Medium / High / Critical
 - Cluster K-Means assigné
- **Recommandation** : message de rétention personnalisé selon le risque

6.3 Routes de l'API

TABLE 12 – Endpoints Flask

Route	Méthode	Description	Retour
/	GET	Interface web principale	HTML
/predict	POST	Prédiction client	JSON
/health	GET	Vérification de santé	JSON

```

1 {
2   "churn_prediction": 1,
3   "churn_probability": 0.7832,
4   "risk_label": "High",
5   "cluster": 1
6 }
  
```

Listing 7 – Exemple de réponse JSON de /predict

6.4 Guide de déploiement

```

1 # 1. Créer et activer l'environnement virtuel
2 python -m venv venv
3 venv\Scripts\activate
4
5 # 2. Installer les dépendances
6 pip install -r requirements.txt
7
8 # 3. Executer le pipeline ML
9 python src/preprocessing.py
10 python src/train_model.py
11
12 # 4. Lancer l'application web
  
```

```
13 python app/app.py
```

Listing 8 – Commandes de déploiement

7 Structure du Projet et Organisation du Code

7.1 Arborescence

```
1 projet_ml_retail/
2 |-- data/
3 |   |-- raw/
4 |   |-- processed/
5 |   \-- train_test/
6 |-- notebooks/
7 |   \-- exploration.ipynb
8 |-- src/
9 |   |-- utils.py
10 |   |-- preprocessing.py
11 |   |-- train_model.py
12 |   \-- predict.py
13 |-- models/
14 |   |-- scaler.joblib
15 |   |-- pca.joblib
16 |   |-- kmeans.joblib
17 |   |-- logistic_regression.joblib
18 |   |-- rf_classifier.joblib
19 |   |-- rf_regressor.joblib
20 |   \-- metadata.joblib
21 |-- app/
22 |   |-- app.py
23 |   \-- templates/index.html
24 |-- reports/
25 |-- requirements.txt
26 |-- README.md
27 \-- .gitignore
```

Listing 9 – Structure complète du projet

7.2 Modules Python

TABLE 13 – Description des modules

Module	Rôle
utils.py	Chargement des données, EDA, visualisations, PCA et métriques
preprocessing.py	Pipeline complet de préparation : parsing, feature engineering, imputation, encodage, scaling et sauvegarde
train_model.py	Entraînement ACP, K-Means, Logistic Regression, Random Forest classifier et regressor
predict.py	Prétraitement d'un nouveau client et prédiction via les modèles chargés
app/app.py	Serveur Flask avec routes GET/POST et chargement des modèles au démarrage

8 Conclusion et Perspectives

8.1 Bilan

Ce projet a couvert l'intégralité de la chaîne de traitement data science appliquée à l'analyse comportementale d'une clientèle e-commerce :

- **Exploration** : identification des valeurs manquantes, des valeurs aberrantes et des features inutiles
- **Préparation** : pipeline structuré réduisant et enrichissant les features
- **Modélisation** : modèles couvrant clustering, classification et régression
- **Déploiement** : API Flask avec interface web fonctionnelle

8.2 Résultats des modèles

TABLE 14 – Récapitulatif des performances

Modèle	Tâche	Métrique	Score
K-Means avec k=2	Clustering	Silhouette	0.594
Régression Logistique	Classification	ROC-AUC	1.000
Random Forest	Classification	ROC-AUC	1.000
RF Régresseur	Régression	R2	voir rapport

8.3 Limites et perspectives

Score parfait : Les scores AUC=1.0 sur le jeu de test suggèrent une possible fuite de données, appelée *data leakage*, liée à des features très fortement corrélées avec le Churn, par exemple `ChurnRiskCategory`. En production, il faudrait supprimer les features dérivées directement de la variable cible.

Les pistes d'amélioration identifiées sont :

1. **Anti-leakage** : supprimer `ChurnRiskCategory`, `RFMSegment` et autres features proxy du churn
2. **SMOTE** : installer `imbalanced-learn` pour rééquilibrer les classes
3. **Optuna** : remplacer `GridSearchCV` par une optimisation bayésienne
4. **SHAP** : ajouter l'explicabilité des prédictions individuelles
5. **MLflow** : versioning et tracking des expériences
6. **Docker** : conteneurisation de l'application Flask